

# Metro-MPI++

Google Summer of Code 2025 Proposal-The FOSSi Foundation

April 8, 2025

## 1 Basic Information

- **Name:** Kislay Arya
- **Email:** kislayarya536@gmail.com
- **GitHub Handle:** kislay536
- **LinkedIn Profile:** Kislay Arya
- **Nationality:** Indian
- **University:** Indian Institute of Technology Kharagpur
- **Degree:** B. Tech (2022-2026)
- **Contact:** (+91)7250067420
- **Timezone:** Indian Standard Time (UTC +5:30)

## 2 About Me

I am a third-year undergraduate student at Indian institute of Technology Kharagpur, pursuing a B.Tech in Electronics and Electrical Communication Engineering with a minor in Computer Science. I am passionate about computer architecture, hardware security and hardware design in general. Over the past year, I have worked on projects like security analysis of modern server grade processors, designing tools to aid the verification process of hardware designs and designing secure systems to mitigate cache based side channel attacks.

Open-source has been instrumental in my learning journey, providing access to real-world systems, simulators, and community-driven innovation. I see GSoC as a unique opportunity to work with experienced mentors and contribute to impactful tools that empower researchers and engineers globally. I want to be a part of GSoC not only to grow technically but also to give back to the open-source communities that have shaped my learning. By contributing to the open-source hardware ecosystem through this project, I aim to create tooling that reduces entry barriers for researchers and students like me, who are eager to explore systems and computer architecture but are limited by free tools available. I believe the work done here will have a lasting impact and further democratize access to advanced simulation environments.

## 3 Why This Project?

With the ever-growing complexity of modern System-on-Chip (SoC) designs and the rise of massively multicore architectures, traditional RTL simulation methodologies often fail to scale effectively. Metro-MPI presents an elegant and practical solution by distributing the simulation workload across MPI processes, enabling substantial performance gains especially in designs with replicated tiles and networks-on-chip.

This project excites me because it sits at the intersection of high-performance simulation, hardware design, and distributed computing—areas I'm deeply passionate about. I have hands-on experience with Verilator, MPI, and RTL designs, and have already begun exploring Metro-MPI by running examples

and understanding its methodology. I see tremendous potential in further improving this framework by integrating it with automatic partitioning, Verilator v5 support, and scaling simulation to thousands of cores.

Moreover, this project aligns closely with my interests in hardware security and computer architecture. Faster and scalable simulations will greatly benefit not just performance studies but also security research, where long runtimes are a major bottleneck. I'm excited about contributing to a tool that lowers this barrier for others in the research and open-source hardware community.

Metro-MPI is also a rare opportunity to contribute to a tool that is both academically relevant and practically impactful, with potential future directions into FPGAs, OpenMP-based parallelism, and speculative rollback mechanisms. I believe my background, motivation, and early engagement make me well-suited to take this project forward under the guidance of the mentors.

## 4 Proposal Description

### 4.1 Overview

As system-on-chip (SoC) designs grow in complexity—especially with manycore architectures—simulation performance becomes a critical bottleneck in the hardware development cycle. RTL simulation is indispensable for accurate verification of digital circuits, but traditional monolithic simulators struggle to scale with the parallelism of modern designs, often leading to prohibitively long simulation times.

Distributed simulation methodologies, such as Metro-MPI, offer a promising solution. By decomposing the hardware design into multiple simulation processes that run in parallel and communicate using MPI (Message Passing Interface), we can exploit the natural parallelism in replicated blocks like processor tiles and network-on-chip (NoC) components. This allows for much faster simulation, especially in manycore environments, without compromising cycle-level accuracy.

#### 4.1.1 Current Framework

The existing Metro-MPI framework relies on **manual partitioning** to enable distributed RTL simulation. In the context of OpenPiton, the default simulation model is manycore-centric and not inherently designed for parallel execution. To overcome this, Metro-MPI introduces two key simulation models: `metro_tile` and `metro_chipset`. These modules are **Verilated separately**, allowing them to be distributed as independent simulation processes.

The MPI-based communication and orchestration across these partitions are managed by temporary top-level files:

- `metro_tile.tmp.v` handles the tile-level simulation logic and supports **multiple instantiations of the same tile module**, making it ideal for manycore architectures.
- `metro_chipset.tmp.v` manages the interconnect, memory, and I/O system simulation in a separate process.

This manual partitioning approach forms the backbone of current Metro-MPI simulations. However, it requires users to explicitly define and maintain the boundaries of each partition, limiting accessibility and flexibility for broader use cases. This project aims to evolve from this manual setup towards **automated or user-guided partitioning**, reducing user overhead and enabling broader adoption.

### 4.2 Objectives

Metro-MPI currently requires **manual partitioning**, which is time-consuming, error-prone, and not easily scalable to large or complex designs. This project aims to address this challenge by:

1. **Automating Partition Detection:** Developing a method—possibly starting with user-guided annotations like pragmas and evolving toward fully automatic detection—to identify candidate top modules for Metro-MPI-based simulation.
2. **Tool Integration:** Embedding Metro-MPI support inside tools like Verilator or Essent, making distributed simulation accessible within widely-used simulation flows.
3. **Enabling Hardware Emulation Use-Cases:** Extending Metro-MPI to work across multiple FPGAs, unlocking the ability to emulate large SoCs across distributed hardware platforms.

### 4.3 Expected Outcome

This project aims to deliver a framework that enables scalable distributed RTL simulation by integrating Metro-MPI with tools like Verilator. Expected outcomes include a semi-automatic partitioning mechanism for identifying and isolating replicable modules (e.g., cores in a tiled architecture), integration of MPI communication into Verilator-generated simulations, and a working demonstration of multi-core RTL simulation using MPI. The project will also produce benchmarks, documentation, and reusable tooling to support future scaling and community adoption.

### 4.4 Proposed Work

The goal of this project is to enable automatic or semi-automatic partitioning of RTL designs for distributed simulation using Metro-MPI, specifically targeting large tiled architectures such as OpenPiton. This involves leveraging existing Verilator features and extending them to support scalable partitioning and interfacing. Verilator has a default multi-threading feature in which the entire design will be divided into MTASKS and then later during simulation they can be simulated parallelly. But this mtasks partition is done on the basis of a cost function to balance the workload on each mtasks. Verilator also supports a feature known as **Hierarchical Verilation**, which allows independent modules to be translated into separate C++ models using metacomments in the RTL code (e.g., `/* verilator hier_block*/`). This approach can be used to identify and isolate tiles—well-encapsulated submodules in manycore systems—by marking them for hierarchical verilation. Although originally intended to parallelize the verilate phase, we can extend this mechanism to support runtime partitioning of simulation as well. This forms the basis for two partitioning approaches:

#### 4.4.1 Active Partitioning (Compiler-Driven)

In the active partitioning approach, the goal is to embed the partitioning process within Verilator itself. This would involve extending Verilator’s elaboration and compilation flow to automatically detect tile-like modules and generate separate simulation models for each. These models would include communication stubs or wrappers to allow inter-process communication using MPI. While the partitioning may initially require user annotations such as metacomments or pragmas, the broader objective is to evolve toward automated partitioning driven by module structure and interface analysis.

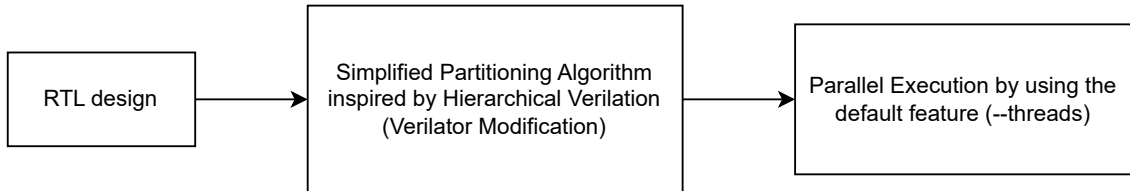


Figure 1: Flow Chart: Active Partitioning

#### 4.4.2 Passive Partitioning (Runtime Framework-Driven)

In contrast, the passive partitioning approach offloads most of the partitioning logic to the simulation framework. In this method, a single tile is independently verilated using the hierarchical verilation feature. Since each tile is a self-contained unit, its IO ports can be connected to external components, such as the NoC router. The chipset (which includes the network-on-chip, memory, and peripheral interfaces) is verilated as a monolithic module, exposing interface ports for connecting tiles. At runtime, the simulation framework instantiates multiple tile models—e.g., 16 tiles for a 4×4 grid—each running in its own MPI process. These instances are dynamically connected to the chipset model through MPI-based message passing. This method reduces the need for intrusive modifications to Verilator and enables rapid prototyping and scaling.

An additional benefit of the passive partitioning strategy is that it naturally aligns with future plans to extend Metro-MPI to support multi-FPGA systems. Since passive partitioning generates multiple independently compiled modules with clean IO boundaries, these modules can be mapped to separate FPGAs in a hardware implementation. The same MPI-based message-passing mechanism used for software simulation can be adapted for high-speed inter-FPGA communication, making it a promising

direction for Metro-MPI@FPGA. This reusability of the simulation-time partitioning infrastructure for real hardware deployment adds long-term value to the approach and increases its practical impact.

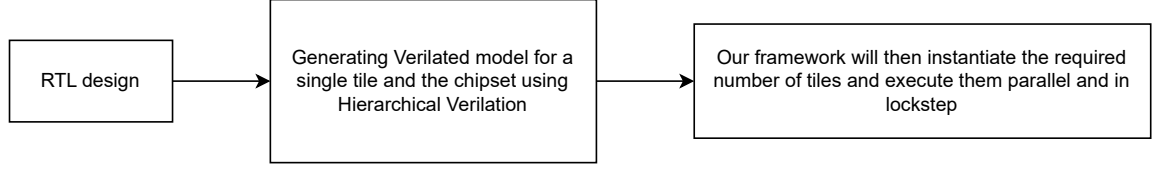


Figure 2: Flow Chart: Passive Partitioning

While hierarchical verilation is a strong starting point, there are limitations that need to be addressed. The current implementation lacks direct support for runtime communication between independently compiled modules, and it assumes relatively simple interface patterns. To overcome these challenges, the project will develop MPI wrapper generators that automate the creation of communication logic at tile boundaries. In addition, a simulation harness will be built to instantiate, connect, and manage multiple tile processes. This harness will handle synchronization, clocking, reset, and cycle-accurate communication between tiles and the chipset. Together, these enhancements will enable scalable distributed simulation with support for thousands of tiles and realistic workloads.

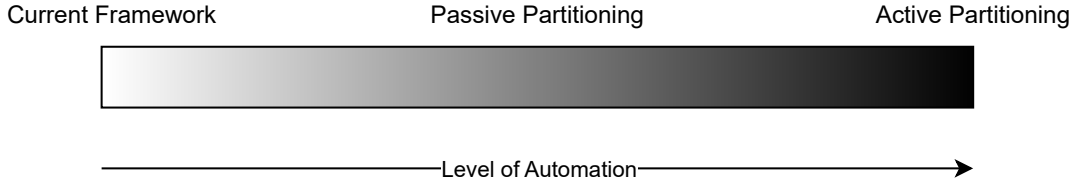


Figure 3: Automation Comparison

## 4.5 Execution Plan

### 4.5.1 Active Partitioning

#### Step 1: Simplifying the Partitioning Algorithm

Verilator’s partitioning algorithm converts an RTL design into a parallelizable task graph by first decomposing the design into fine-grained macro-tasks (MTasks), each representing discrete logic operations (e.g., assignments, always blocks) with dependencies modeled as edges. These MTasks are iteratively merged into coarser units using critical path (CP) analysis—prioritizing merges that minimize CP elongation—to balance thread utilization and reduce synchronization overhead. The algorithm avoids cycles to maintain a directed acyclic graph (DAG) structure and resolves data hazards (e.g., shared variable accesses) by grouping interdependent tasks. Finally, a static scheduler assigns merged MTasks to threads, enforcing dependency edges to ensure semantic correctness during parallel simulation. This optimizes CPU core usage while preserving the functional behavior of the original RTL design.

This approach decomposes the RTL design into hierarchical blocks (e.g., independent tiles with NoC routers) that operate in parallel but synchronize for inter-tile communication. First, each tile is treated as a coarse-grained MTask, with computational costs derived hierarchically from its internal logic (e.g., core operations, local cache). Threads simulate tiles concurrently during independent execution phases, processing internal logic without synchronization. Second, at predefined lock-step intervals, threads pause to resolve NoC transactions (e.g., packet routing, buffer updates), ensuring deterministic communication between tiles. Dependencies are restricted to these synchronization points, avoiding fine-grained coordination while maintaining NoC coherence. Thread assignments prioritize spatial locality (e.g., grouping tiles sharing NoC subnets), balancing parallelism with synchronization overhead. This method optimizes scalability for multi-tile designs, leveraging hierarchical isolation for parallel execution while enforcing synchronization only during NoC interaction phases.

To facilitate user-driven selection of the active partitioning approach, a new Verilator command-line flag `--active` will be introduced. This flag will trigger the use of a simplified version of the partitioning algorithm that specifically targets tile-based designs with predictable interconnect structures, such as manycore systems. This streamlined algorithm will be designed to rapidly generate parallelizable models using the hierarchical tiling and lock-step communication assumptions discussed above.

The current full-featured partitioning algorithm will be preserved to maintain backward compatibility and generality. The simplified version will only be invoked when the `--active` option is used, offering faster compilation times and reduced analysis overhead for supported design patterns.

### Step 2: Embedding the MPI Framework

Once the RTL design is partitioned into parallel simulation units—either through active compiler-driven partitioning or passive runtime-based partitioning—the next crucial step is to embed the MPI (Message Passing Interface) framework within the simulation infrastructure. This step enables distributed execution of the simulation by facilitating communication and synchronization between the isolated tile processes and the monolithic chipset or other tile modules.

The key objective is to establish seamless and cycle-accurate communication across MPI processes, mirroring the original on-chip communication semantics (e.g., NoC transactions, memory requests/responses, interrupts). This step involves both compile-time instrumentation of the generated simulation models and runtime orchestration of the distributed processes.

To achieve communication across partitioned modules, each tile and the chipset must be equipped with MPI-based stubs that encode and decode communication packets. These stubs serve as stand-ins for the original on-chip wires and buses that are now distributed across processes.

- **Generation of MPI Wrappers:** A wrapper generator will be developed to automatically produce C++ classes/functions that wrap standard I/O ports (e.g., flit signals, handshake lines, memory buses) with equivalent MPI `Send` and MPI `Recv` calls. These wrappers will support:
  - Serialization and deserialization of signal bundles (e.g., structs representing flits or memory requests)
  - Blocking and non-blocking modes to match the RTL’s handshaking semantics
  - Cycle-level synchronization via barrier-style coordination where required
- **Interface Matching:** The wrapper generator will infer the I/O interface of each module (via Verilator’s introspection or user-provided annotations) and automatically map it to corresponding MPI message formats.

## 4.5.2 Passive Partitioning

Passive partitioning aims to reduce inter-process communication overhead by grouping non-communicating modules into single simulation processes, while still leveraging the Metro-MPI methodology of separate Verilated models for tiles and chipsets. This approach will integrate with the existing partitioning pipeline while preserving flexibility and modularity. The following steps outline the planned implementation:

- **Step 1: Explore and analyze current simulation models**
  - Study the existing manually partitioned framework, focusing on how `metro_tile.tmp.v` and `metro_chipset.tmp.v` define the communication boundaries.
  - Understand how MPI is used between these modules and identify opportunities for grouping modules that do not engage in external communication.
- **Step 2: Merge passive partitioning logic into the current partitioning algorithm**
  - Extend the automatic partitioning pipeline (as discussed in Section 4.5.1) to detect passive modules using structural analysis.
  - Instead of generating a single unified Verilator model, maintain the current structure with **two separate models** (`tile` and `chipset`) — this preserves the manual partitioning design, but enhances it with automated passive grouping within each model.
- **Step 3: Implement the `--passive` option in Verilator**
  - Add a new Verilator CLI option `--passive` that triggers the generation of the `tile` and `chipset` Verilated models using passive partitioning logic.

- Create a new source module within Verilator, `V3MPI.cpp` and `V3MPI.h`, responsible for generating and managing communication code between the two models in passive mode.
- This module will abstract away MPI interface details and enable passive-aware scheduling and synchronization between simulation units.
- **Step 4: Integrate, test, and validate the passive mode**
  - Ensure that the `--passive` mode is compatible with OpenPiton and other Metro-MPI-compatible designs.
  - Compare simulation correctness, runtime, and communication overhead with the baseline Metro-MPI and the unified auto-partitioned model.
  - Test across multiple configurations (e.g., 64, 256, and 1024 tiles) to evaluate scalability and efficiency improvements.

## 5 Project Timeline

### 5.1 Project Timeline and Deliverables

The project will be executed in structured phases, each focusing on a key aspect of the active and passive partitioning support in Metro-MPI++. The following breakdown outlines the major goals, deliverables, and implementation details for each phase:

#### Pre-GSoC and Community Bonding Period (April 30 – June 1)

- Study the current Metro-MPI++ infrastructure, including how `metro_tile.tmp.v` and `metro_chipset.tmp.v` are generated.
- Explore Verilator’s existing partitioning algorithm and understand the construction and scheduling of MTasks.
- Analyze MPI-based communication and synchronization models currently used.
- Design the CLI flags `--active` and `--passive` to support the proposed dual-mode simulation model.
- Draft architecture for a new source file `V3MPI.cpp/.h` that will handle MPI communication in passive partitioning mode.

#### Coding Phase I (June 2 – July 13)

- Implement the `--passive` flag support in Verilator to generate separate models for tiles and chipset.
- Develop `V3MPI.cpp/.h` to support MPI-based communication:
  - Serialization/deserialization of signal bundles.
  - Blocking/non-blocking MPI send/recv operations.
  - Cycle-level synchronization via MPI barriers.
- Automatically generate MPI wrappers for inter-process communication over flits, handshake signals, and memory buses.
- Test passive-mode simulation using small multi-tile OpenPiton designs.

#### Mid-Term Evaluation (July 14 – July 18)

- Validate correctness of passive-mode simulations.
- Perform initial performance benchmarking and debugging.

## **Coding Phase II (July 14 – August 25)**

- Build a simplified version of Verilator’s partitioning algorithm to be used when `--active` is set.
- Use hierarchical decomposition (e.g., tiles as coarse-grained MTasks) to enable scalable multi-threaded simulation.
- Integrate the simplified algorithm into Verilator while preserving the existing partitioning logic for the default flow.
- Enable lock-step synchronization at NoC boundaries while maximizing parallel execution during independent computation phases.
- Integrate both `--passive` and `--active` flows with Metro-MPI++’s runtime infrastructure.
- Conduct performance tests across both modes (active and passive) with increasing tile counts.

## **Final Evaluation (August 25 – September 1)**

- Final testing across multiple configurations and communication topologies.
- Comprehensive documentation:
  - Usage guide for `--active` and `--passive`.
  - Overview of V3MPI and its integration with Verilator.
  - Example workflows and debugging instructions.
- Prepare the final report and submit deliverables.

## 5.2 Week-wise Plan

| Period                                       | Objective                                                                                                      | Difficulty |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------|
| <b>Pre-GSoC period</b><br>(April 30 - May 7) | Explore Metro-MPI++ flow, Verilator integration, and coherence mechanism in OpenPiton                          | ●          |
| <b>Week 1</b><br>(May 8 - May 14)            | Understand Verilator’s partitioning (MTask, comms)<br>Begin CLI design for –passive and –active                | ●          |
| <b>Week 2</b><br>(May 15 - May 21)           | Analyze Metro-MPI++ MPI logic; design comm layer<br>Begin drafting V3MPI.cpp/.h architecture                   | ●          |
| <b>Week 3</b><br>(May 22 - May 28)           | Review how <code>metro_tile.tmp.v</code> is created<br>Finalize –passive CLI parsing and plan implementation   | ●          |
| <b>Week 4</b><br>(May 29 - June 4)           | Prototype CLI parsing and passive-mode setup<br>Study flit serialization format                                | ●          |
| <b>Week 5</b><br>(June 5 - June 11)          | Implement –passive code path in Verilator<br>Start writing V3MPI send/receive wrappers                         | ●          |
| <b>Week 6</b><br>(June 12 - June 18)         | Build MPI layer for communication of memory/flit packets<br>Create utilities for serialization/deserialization | ●          |
| <b>Week 7</b><br>(June 19 - June 25)         | Test passive mode simulation on basic topologies<br>Debug data correctness across MPI interfaces               | ●          |
| <b>Week 8</b><br>(June 26 - July 2)          | Extend passive mode to support tile+chipset flows<br>Evaluate performance and correctness                      | ●          |
| <b>Week 9</b><br>(July 3 - July 9)           | Refine passive-mode interface and resolve bugs<br>Prepare for mid-term evaluation                              | ●          |
| <b>Week 10</b><br>(July 10 - July 16)        | Start working on simplified partitioning logic<br>Design coarse-grained MTasks for –active mode                | ●          |
| <b>Week 11</b><br>(July 17 - July 23)        | Implement NoC synchronization and handshake logic<br>Integrate with existing Verilator infrastructure          | ●          |
| <b>Week 12</b><br>(July 24 - July 30)        | Finalize –active CLI and wire into build pipeline<br>Start testing basic multicore setups with active mode     | ●          |
| <b>Week 13</b><br>(July 31 - August 6)       | Run correctness and equivalence tests<br>Compare functional parity with passive mode                           | ●          |
| <b>Week 14</b><br>(August 7 - August 13)     | Run performance benchmarks for both modes<br>Profile bottlenecks in NoC and serialization                      | ●          |
| <b>Week 15</b><br>(August 14 - August 20)    | Finalize all features and fix edge-case bugs<br>Polish V3MPI and comm-layer API                                | ●          |
| <b>Week 16</b><br>(August 21 - August 25)    | Prepare documentation, guides, and diagrams<br>Submit final code and blog write-up                             | ●          |

Table 1: Detailed Week-wise GSoC Work Plan

## 6 Availability

I will be available full-time during my summer break from May 1 to July 14 and plan to dedicate 30–35 hours per week to the project during this period. Once my academic semester begins, my availability will reduce slightly, and I will continue contributing around 20 hours per week.

To account for the transition period from July 14 to July 22—when I’ll be settling into the new semester—I have planned to start working earlier and make substantial progress in the initial weeks. This will help mitigate any impact on the overall timeline. I will also remain in regular contact with my mentors and adjust my schedule if any unforeseen events arise, ensuring steady progress throughout the program.